LAB: Grayscale Image Segmentation - Gear

Date: 2026-04-02 **Author:** lee jeayong(22000561)

Introduction

1. Objective

The goal of this lab is to develop a machine vision system that can inspect defective plastic gears. We are asked to segment the target objects from the background and determine the number of defective teeth and quality by applying thresholding, morphology, and contour analysis. Goal:count the number of teeth of gear and detect broken teeth.

The system will calculate the following:

- Number of defective teeth
- Diameter of the gear
- Quality Inspection (Pass or Fail)

2. Preparation

Software Installation

- OpenCV, C++

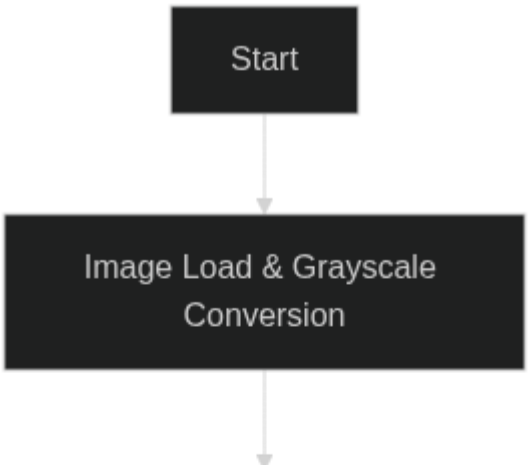
Dataset

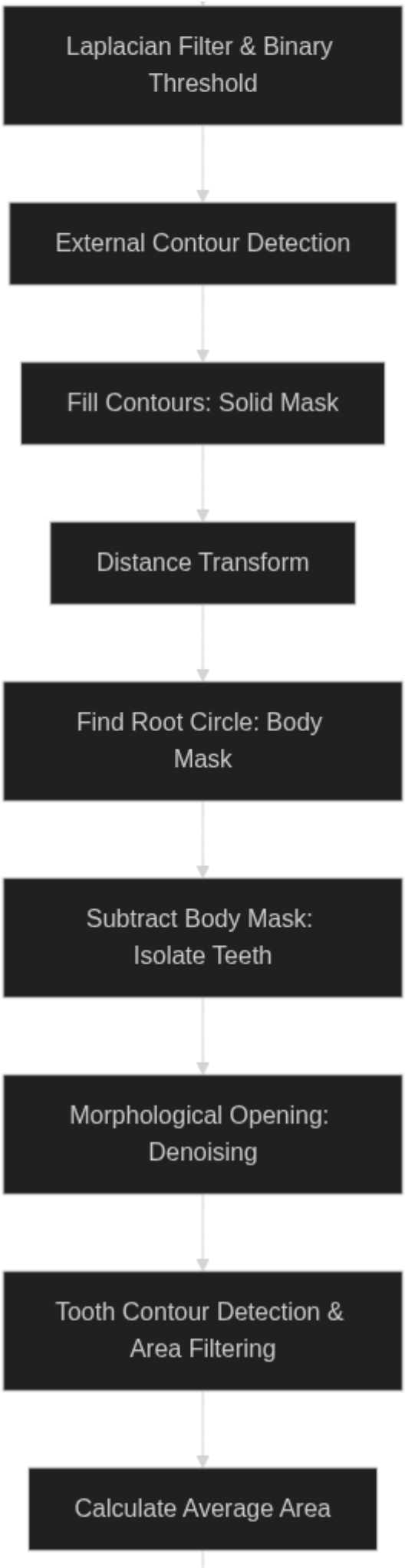
- `Lab_GrayScale_Gears.zip` (Gear1.jpg, Gear2.jpg, Gear3.jpg, Gear4.jpg)

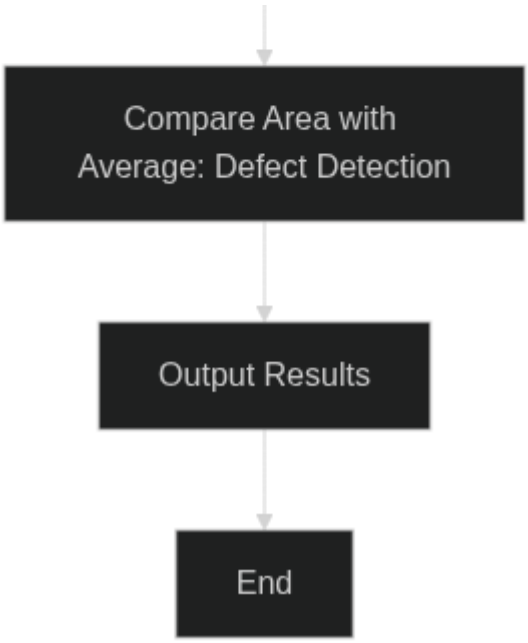
Algorithm

1. Overview

The algorithm follows these main steps:




Laplacian Filter & Binary Threshold
External Contour Detection
Fill Contours: Solid Mask
Distance Transform
Find Root Circle: Body Mask
Subtract Body Mask: Isolate Teeth
Morphological Opening: Denoising
Tooth Contour Detection & Area Filtering
Calculate Average Area



1. **Image Load & Grayscale Conversion:** Read the image and convert it to grayscale.
2. **Edge Extraction:** Apply a Laplacian filter to extract edges and threshold it to a binary image.
3. **Solid Gear Masking:** Find boundaries of the object and fill the contours to create a solid mask of the entire gear.
4. **Inner Body Separation:** Apply Distance Transform to the solid mask to find the center and the root radius of the gear. Create a mask for the inner circular body.
5. **Teeth Extraction:** Subtract the inner body mask from the solid gear mask to isolate the teeth. Apply morphological opening to denoise.
6. **Defect Detection:** Find contours of the isolated teeth, calculate their areas, and compute the trimmed average area (excluding top 3 and bottom 3 extremes). A tooth is considered defective if its area is less than 90% or more than 110% of the trimmed average area (i.e., broken or missing).

2. Procedure

Edge Detection

Since the gears share similar intensity values with some parts of the background, detecting edges directly is more robust than simple thresholding. Thus, a `Laplacian` filter (kernel size 3) is applied to the grayscale image. The result is binarized using a threshold value of 30.

Contour and Solid Mask

`findContours` is applied to the binarized edges to find the external boundaries. The contours are then filled using `drawContours` with `FILLED` mode to generate a solid mask covering the entire gear. This procedure is done to isolate the complete physical shape of the gear from the complex background and its internal holes, providing a solid foreground object essential to separate the teeth in the later steps.

Distance Transform

To isolate the teeth from the gear body, we find the inscribed circle of the gear's inner body. `distanceTransform` is applied to the solid mask. Using `minMaxLoc`, the pixel with the maximum distance to

the background is found, which corresponds to the gear's center, and its value corresponds to the inner radius. This allows building a circular mask representing the body.

Defective Teeth Determination

The inner body mask is subtracted from the solid gear mask, leaving only the gear teeth. `morphologyEx` (MORPH_OPEN) removes remaining noise. Consequent `findContours` groups each tooth individually. The areas of all valid teeth contours (>15.0) are measured using `contourArea`. To avoid outliers, the top 3 and bottom 3 area values are excluded, and the trimmed average area (중간 평균 면적) is calculated from the remaining values. If a contour's area falls below 90% or exceeds 110% of this trimmed average area, it is flagged as a defective tooth.

Result and Discussion

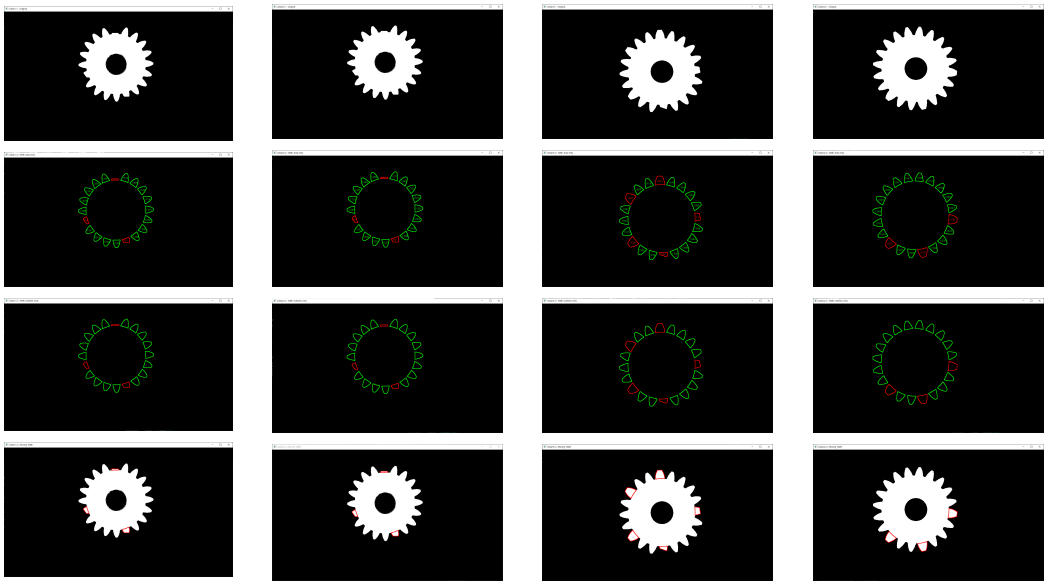
1. Final Result

For each gear image, the program successfully outputs:

- Original image
- Teeth Area Only image (showing segmented teeth and their areas)
- Teeth Outlines Only image (showing normal and defective teeth outlines)
- Missing Teeth image (original image highlighting the defects)

In the terminal, the algorithm prints the number of detected teeth, the trimmed average area of the teeth, and the broken/defective teeth count.

Analysis Results

	Sample #1	Sample #2	Sample #3	Sample #4
Output Images				
Teeth numbers	20	20	20	20
Trimmed Avg. Area	1295.2	1295.8	1578.8	1546.1

	Sample #1	Sample #2	Sample #3	Sample #4
Defective Teeth	3	3	5	3
Quality	FAIL	FAIL	FAIL	FAIL

2. Discussion

The proposed algorithm robustly segments the plastic gear using a Laplacian filter and Distance Transform, making it independent of exact lighting conditions that might hinder simple global thresholding. Using a trimmed average area of the teeth (excluding the largest and smallest areas) successfully isolates the normal teeth size from outliers, effectively adapting the defect threshold for different gear sizes while ignoring completely broken teeth during the average calculation.

Conclusion

The machine vision system successfully detected defective teeth in the given plastic gear images. Utilizing edge detection, morphological processing, and distance transformation, this algorithm successfully separated the gear teeth from the body and identified the defective teeth based on their individual area sizes. The results were also satisfactory, showing no significant difference compared to human visual inspection.

Appendix

main.cpp

```
#include <opencv2/opencv.hpp>
#include <iostream>
#include <iomanip>
#include <vector>
#include <string>
#include <algorithm>

using namespace cv;
using namespace std;

// ==================== Function Declarations ====================

// Apply Laplacian edge detection and return binary edge map
Mat detectEdges(const Mat& img_gray);

// Fill external contours to create a solid gear mask
Mat createSolidMask(const Mat& edges_binary);

// Separate gear body from teeth using distance transform
void separateBodyAndTeeth(const Mat& mask_solid, Mat& mask_teeth);

// Filter small-noise contours and collect valid teeth contours/areas
```

```
void filterTeethContours(const vector<vector<Point>>& all_contours,
                         vector<vector<Point>>& contours,
                         vector<double>& areas,
                         double min_area);

// Compute trimmed mean (exclude top/bottom N values)
double calcTrimmedAverage(const vector<double>& areas, int trim_count);

// Draw results on canvases and count defects
int drawResults(const vector<vector<Point>>& contours,
                double min_area, double max_area,
                Mat& img_res2, Mat& img_res3, Mat& img_res4);

// Print analysis table to console
void printResult(const string& file_name, int teeth_count,
                 double avg_area, int defect_count);

// ========================== main() ============================

int main() {
    vector<string> file_names = { "../../Image/Gear1.jpg",
"../../Image/Gear2.jpg", "../../Image/Gear3.jpg", "../../Image/Gear4.jpg" };

    for (int idx = 0; idx < (int)file_names.size(); idx++) {
        string file_name = file_names[idx];
        string prefix = "[Gear" + to_string(idx + 1) + "] ";

        // 1. Load image
        Mat img_color = imread(file_name, IMREAD_COLOR);
        if (img_color.empty()) {
            cerr << prefix << "Could not load image." << endl;
            continue;
        }

        Mat img_gray;
        cvtColor(img_color, img_gray, COLOR_BGR2GRAY);

        // 2. Edge detection
        Mat edges_binary = detectEdges(img_gray);

        // 3. Create solid gear mask
        Mat mask_solid = createSolidMask(edges_binary);

        // 4. Separate body and teeth
        Mat mask_teeth;
        separateBodyAndTeeth(mask_solid, mask_teeth);

        // 5. Extract valid teeth contours
        vector<vector<Point>> all_teeth_contours;
        findContours(mask_teeth, all_teeth_contours, RETR_EXTERNAL,
CHAIN_APPROX_SIMPLE);

        vector<vector<Point>> contours;
        vector<double> areas;
```

```
        filterTeethContours(all_teeth_contours, contours, areas, 15.0);

        // 6. Compute trimmed average and defect thresholds
        double avg_area = calcTrimmedAverage(areas, 3);
        double min_normal_area = avg_area * 0.9;
        double max_normal_area = avg_area * 1.1;

        // 7. Render results
        Mat img_res2 = Mat::zeros(img_color.size(), CV_8UC3);
        Mat img_res3 = Mat::zeros(img_color.size(), CV_8UC3);
        Mat img_res4 = img_color.clone();

        int defect_count = drawResults(contours, min_normal_area, max_normal_area,
                                       img_res2, img_res3, img_res4);

        // 8. Print analysis table
        printResult(file_name, (int)contours.size(), avg_area, defect_count);

        // 9. Display windows
        imshow(prefix + "1. Original", img_color);
        imshow(prefix + "2. Teeth Area Only", img_res2);
        imshow(prefix + "3. Teeth Outlines Only", img_res3);
        imshow(prefix + "4. Missing Teeth", img_res4);
    }

    cout << "\n>> All analyses completed. Press any key to exit..." << endl;
    waitKey(0);

    return 0;
}

// ==================== Function Definitions ======================

Mat detectEdges(const Mat& img_gray) {
    Mat img_laplacian, edges_binary;
    Laplacian(img_gray, img_laplacian, CV_16S, 3);
    convertScaleAbs(img_laplacian, img_laplacian);
    threshold(img_laplacian, edges_binary, 30, 255, THRESH_BINARY);
    return edges_binary;
}

Mat createSolidMask(const Mat& edges_binary) {
    vector<vector<Point>> edge_contours;
    findContours(edges_binary, edge_contours, RETR_EXTERNAL, CHAIN_APPROX_SIMPLE);

    Mat mask_solid = Mat::zeros(edges_binary.size(), CV_8U);
    drawContours(mask_solid, edge_contours, -1, Scalar(255), FILLED);
    return mask_solid;
}

void separateBodyAndTeeth(const Mat& mask_solid, Mat& mask_teeth) {
    // Distance transform to find gear center and root radius
    Mat dist;
    distanceTransform(mask_solid, dist, DIST_L2, 5);
```

```
    double minVal, maxVal;
    Point minLoc, maxLoc;
    minMaxLoc(dist, &minVal, &maxVal, &minLoc, &maxLoc);

    // Create circular body mask
    Mat mask_body = Mat::zeros(mask_solid.size(), CV_8U);
    int root_radius = cvRound(maxVal) + 1;
    circle(mask_body, maxLoc, root_radius, Scalar(255), FILLED);

    // Subtract body from solid to get teeth region
    subtract(mask_solid, mask_body, mask_teeth);

    // Remove small noise with morphological opening
    Mat kernel = getStructuringElement(MORPH_RECT, Size(3, 3));
    morphologyEx(mask_teeth, mask_teeth, MORPH_OPEN, kernel);
}

void filterTeethContours(const vector<vector<Point>>& all_contours,
                         vector<vector<Point>>& contours,
                         vector<double>& areas,
                         double min_area) {
    for (size_t i = 0; i < all_contours.size(); i++) {
        double area = contourArea(all_contours[i]);
        if (area > min_area) {
            contours.push_back(all_contours[i]);
            areas.push_back(area);
        }
    }
}

double calcTrimmedAverage(const vector<double>& areas, int trim_count) {
    if (areas.empty()) return 0.0;

    vector<double> sorted_areas = areas;
    sort(sorted_areas.begin(), sorted_areas.end());

    // Skip trimming if not enough samples
    if ((int)sorted_areas.size() <= trim_count * 2)
        trim_count = 0;

    int valid_count = (int)sorted_areas.size() - (trim_count * 2);
    double sum = 0.0;

    for (int i = trim_count; i < (int)sorted_areas.size() - trim_count; i++)
        sum += sorted_areas[i];

    return valid_count > 0 ? sum / valid_count : 0.0;
}

int drawResults(const vector<vector<Point>>& contours,
                double min_area, double max_area,
                Mat& img_res2, Mat& img_res3, Mat& img_res4) {
    int defect_count = 0;
```

```
    for (size_t i = 0; i < contours.size(); i++) {
        double area = contourArea(contours[i]);
        bool is_defect = (area < min_area || area > max_area);

        if (is_defect) defect_count++;

        Scalar color = is_defect ? Scalar(0, 0, 255) : Scalar(0, 255, 0);

        // Draw on labeled canvas and outline-only canvas
        drawContours(img_res2, contours, (int)i, color, 2);
        drawContours(img_res3, contours, (int)i, color, 2);

        // Mark defects on original image
        if (is_defect)
            drawContours(img_res4, contours, (int)i, Scalar(0, 0, 255), 2);

        // Label area value at contour center
        Moments M = moments(contours[i]);
        if (M.m00 != 0) {
            int cX = (int)(M.m10 / M.m00);
            int cY = (int)(M.m01 / M.m00);
            putText(img_res2, to_string((int)area),
                    Point(cX - 15, cY + 5),
                    FONT_HERSHEY_SIMPLEX, 0.4, color, 1);
        }
    }

    return defect_count;
}

void printResult(const string& file_name, int teeth_count,
                 double avg_area, int defect_count) {
    cout << "\n\n[" << file_name << "] Analysis Result" << endl;
    cout << "=========================================" << endl;
    cout << "| " << left << setw(10) << " Teeth Cnt"
         << "| " << setw(12) << " Trimmed Avg"
         << "| " << setw(12) << "Defects Cnt" << " |" << endl;
    cout << "-----------------------------------------" << endl;
    cout << "| " << left << setw(10) << teeth_count
         << "| " << setw(12) << fixed << setprecision(1) << avg_area
         << "| " << setw(12) << defect_count << " |" << endl;
    cout << "=========================================" << endl;
}
```